

04-mouse-keyboard-actions

Mouse, Keyboard Actions in Playwright

In Playwright, mouse and keyboard actions are available directly on the `locator` (high level) or on `page.mouse` / `page.keyboard` (low level). The common actions are:

- `hover`
- `hover and click`
- `right click`
- `double click`
- `drag and drop`
- `click and hold`
- `keyboard actions`

*Note: Unlike Selenium, Playwright does **not** have an `Actions` builder class. Each action is its own auto-waiting method on the locator, so you don't need a `perform()` call at the end.*

Mouse Hover

Used to hover over a dropdown, tooltip, or help text so it becomes visible.

```
await page.locator("#help").hover();
```

Right Click

Right click on any element using the `button` option.

```
await page.locator("#rightClickBtn").click({ button: "right" });
```

The `button` option accepts:

- `left` (default)
- `right`
- `middle`

Double Click

To perform a double click we use the `dblclick()` method.

```
await page.locator("#db-click").dblclick();
```

Drag and Drop

Playwright provides a built-in method `source.dragTo(target)`. There is also a low-level manual approach using `page.mouse`.

```
const source = page.locator("#src");
const target = page.locator("#trg");

// Approach 01 - manual (low level)
await source.hover();
await page.mouse.down(); // press mouse on source

await target.hover();
await page.mouse.up(); // release on target

// Approach 02 - built in (preferred)
await source.dragTo(target);
```

Keyboard actions

Playwright handles keys with `page.keyboard.press()`. Shortcuts are written with `+`.

```
// Shortcut / multiple keys together
await page.keyboard.press("Control+A");
await page.keyboard.press("Shift+A");

// Single key
await page.keyboard.press("a");
await page.keyboard.press("Enter");

// Type a whole string character by character (fires individual key events)
// `type()` is deprecated, use `pressSequentially()` instead
await page.locator("#search").pressSequentially("kundalik", { delay: 100 });
```

`press()` is one key/shortcut at a time. `pressSequentially()` types a full string character by character and fires real key events (the `delay` option, in ms, adds a pause between each key — useful for inputs that react on every keystroke). It replaces the deprecated `type()`. To fill a field quickly without per-key events, prefer `locator.fill("text")`.

Actions in Selenium Java

In Selenium all these actions are performed using the `Actions` class.

```
Actions act = new Actions(driver);
```

- The `Actions` class supports **method chaining**.
- It follows the **builder pattern**.
- We can chain multiple actions together, but to actually run them we must call `perform()` at the end.
- Chaining makes complex operations readable and efficient.

*Note: each block below is a **standalone example** of one gesture. If you ever run several gestures back-to-back on the **same** `act` instance, remember Selenium accumulates queued actions and `perform()` does not clear them — so use a fresh `Actions` instance per gesture in real tests. `Duration.ofSeconds(. )` needs `import java.time.Duration;`*

```
WebDriver driver = new ChromeDriver();
Actions act = new Actions(driver);

WebElement element = driver.findElement(By.id("help"));

// Mouse hover
act.moveToElement(element)
  .perform();

// Hover and click
act.moveToElement(element)
  .click()
  .perform();

// Right click
act.contextClick(element)
  .perform();

// Double click
act.doubleClick(element)
  .perform();

// Click and hold
act.clickAndHold(element)
  .perform();

// Click, hold, then release
act.clickAndHold(element)
  .pause(Duration.ofSeconds(2))
  .release()
  .perform();
```

Drag and Drop

To drag and drop we use the `dragAndDrop(source, target)` method with the source and target elements.

```
WebElement source = driver.findElement(By.id("src"));
WebElement target = driver.findElement(By.id("trg"));

act.dragAndDrop(source, target)
    .perform();
```

Sometimes `dragAndDrop(src, trg)` won't work because of custom JS-based drag scripts that Selenium's method cannot capture. In that case use the manual click-hold-move-release approach:

```
WebElement source = driver.findElement(By.id("src"));
WebElement target = driver.findElement(By.id("trg"));

act.clickAndHold(source)
    .moveToElement(target)
    .release()
    .perform();
```

Move using co-ordinates

To drag by a pixel offset instead of to a target element:

```
act.dragAndDropBy(source, xOffset, yOffset)
    .perform();
```

Here `xOffset` and `yOffset` are the distances to move in `px`.

KeyPress in Selenium

The `Keys` enum is used to perform keyboard operations in Selenium.

```
import org.openqa.selenium.Keys;

WebElement searchLoc = driver.findElement(By.name("q"));

act.sendKeys(searchLoc, "kundalik")
    .sendKeys(Keys.ENTER)
    .perform();
```

- `keyDown()` ⇒ simulates pressing and holding a key.
- `keyUp()` ⇒ simulates releasing the held key.

Performing Ctrl + A

```
act.keyDown(Keys.CONTROL) // press & hold Ctrl
    .sendKeys("a")         // press A
    .keyUp(Keys.CONTROL)  // release Ctrl
    .perform();
```

Interview Questions

Q - Why do we use `keyDown` before `keyUp`?

Ans:-

- `keyDown()` presses and holds the key.
- `keyUp()` releases the held key.
- This mimics how a real user interacts: they press and hold the first key, press the second key, then release. Without `keyUp`, the key stays logically held down.

Q - Why does Selenium need `perform()` but Playwright does not?

Ans:-

- Selenium's `Actions` uses the builder pattern — it only builds a queue of actions; `perform()` actually executes them.
- Playwright actions are individual auto-waiting methods that execute immediately when awaited, so no `perform()` is needed.

Q - Difference between `dragTo` / `dragAndDrop` and the manual approach?

Ans:-

- The built-in method is one line and works for standard HTML drag.
- The manual approach (hover/down/move/up) is used when the built-in method fails on custom JS drag-and-drop implementations.

Q - `hover()` use cases?

Ans:-

- Revealing hidden menus, dropdowns, tooltips, or help text that only appear on hover.

Action Methods Comparison

Action	Playwright	Selenium (Java)
Hover	<code>locator.hover()</code>	<code>act.moveToElement(el).perform()</code>
Hover + click	<code>locator.hover()</code> then <code>locator.click()</code>	<code>act.moveToElement(el).click().perform()</code>
Right click	<code>locator.click({ button: "right" })</code>	<code>act.contextClick(el).perform()</code>
Double click	<code>locator.dblclick()</code>	<code>act.doubleClick(el).perform()</code>
Click and hold	<code>page.mouse.down()</code>	<code>act.clickAndHold(el).perform()</code>
Release	<code>page.mouse.up()</code>	<code>act.release().perform()</code>

Action	Playwright	Selenium (Java)
Drag and drop	<code>source.dragTo(target)</code>	<code>act.dragAndDrop(src, trg).perform()</code>
Move by offset	<code>page.mouse.move(x, y)</code>	<code>act.dragAndDropBy(src, x, y).perform()</code>
Press key / shortcut	<code>page.keyboard.press("Control+A")</code>	<code>act.keyDown(Keys.CONTROL).sendKeys("a").keyUp(Keys.CONTROL).perform()</code>
Type string	<code>locator.pressSequentially("text", { delay: 100 })</code>	<code>act.sendKeys("text").perform()</code>
Needs perform()	No (auto executes)	Yes (builder pattern)

Keys Comparison

Selenium	Playwright
<code>Keys.ENTER</code>	<code>"Enter"</code>
<code>Keys.CONTROL</code>	<code>"Control"</code>
<code>Keys.SHIFT</code>	<code>"Shift"</code>
<code>Keys.TAB</code>	<code>"Tab"</code>
<code>Keys.ESCAPE</code>	<code>"Escape"</code>
<code>Keys.ARROW_DOWN</code>	<code>"ArrowDown"</code>